

Name : Bisma Fatima

Class : CS- 24 - A

Roll No : F24605036

Course : Data Structure

Code : CS- 210

Lecturer : Sir Mehran Yousaf

Assignment 02

Name : Bisma Fatima

Question No 01

Evaluate a postfix expression also known as
Reverse Polish Notation-

Program

```
#include <iostream>
```

```
#include <cctype>
```

```
using namespace std;
```

```
class Stack {
```

```
    private:
```

```
        int arr[100];
```

```
        int top;
```

```
    public:
```

```
        stack()
```

```
        { top = -1;
```

```
        }
```

```
        bool isempty()
```

```
        { return top == -1;
```

```
        }
```

```
        bool isfull()
```

```
        {
```

```
            return top == 99;
```

```
        }
```

```
void push(int value)
{
    if (isfull())
    {
        cout << "Stack Overflow!";
        return;
    }
    arr[++top] = value;
}
```

```
int pop()
{
    if (isempty())
    {
        cout << "Stack is empty!";
        return -1;
    }
    return arr[top--];
}
```

```
int peek()
{
    if (isempty())
    {
        cout << "Stack is empty!";
        return -1;
    }
    return arr[top];
}
```

```
};
```

Name: Bisma Fatima

```
int evaluatePostfix (const string &expr)
```

```
{
    Stack s;
    int i = 0;
    while (i < expr.length())
    {
        char ch = expr[i];
        if (ch == ' ')
        {
            i++;
            continue;
        }
        if (isdigit(ch))
        {
            int num = 0;
            while (i < expr.length() && isdigit(expr[i]))
            {
                num = num * 10 + (expr[i] - '0');
                i++;
            }
            s.push(num);
        }
        else
        {
            int val2 = s.pop();
            int val1 = s.pop();
            int result = 0;

```

```
switch(ch)
```

```
{
```

```
    case '+':
```

```
        result = val1 + val2;
```

```
        break;
```

```
    case '*':
```

```
        result = val1 * val2; break;
```

```
    case '-':
```

```
        result = val1 - val2; break;
```

```
    case '/':
```

```
        result = val1 / val2;
```

```
        break;
```

```
    default:
```

```
        cout << "Invalid operator" << ch;
```

```
        return -1;
```

```
}
```

```
    s.push(result);
```

```
    it++;
```

```
}
```

```
}
```

```
    return s.pop();
```

```
}
```

```
int main()
```

```
{
```

```
    string expr = "512+4*+3-";
```



```
cout << "Postfix expression" << expr;
```

```
int result = evaluate_Postfix(expr);
```

```
cout << "Result : " << result;
```

```
}
```

Dry Run

Postfix expression:

5 12 + 4 * + 3 -

Step by step evaluation:

1. 5 is pushed to the stack : [5]
2. 1 is pushed to the stack : [5, 1]
3. 2 is pushed to the stack : [5, 1, 2]
4. + operator is encountered:
 - Pop 2 and 1 from the stack : [5]
 - calculate $2 + 1 = 3$
 - Push 3 to the stack : [5, 3]
5. 4 is pushed to the stack : [5, 3, 4]
6. * operator is encountered:
 - pop 4 and 3 from the stack : [5]
 - calculate $4 * 3 = 12$
 - push 12 to the stack : [5, 12]
7. + operator encountered:

- pop 12 and 5 from the stack: $[]$
- calculate $5 + 12 = 17$
- push 17 to the stack: $[17]$

8. 3 is pushed to the stack: $[17, 3]$

9. - operator is encountered:

- pop 3 and 17 from the stack: $[]$
- calculate $17 - 3 = 14$
- push 14 to the stack: $[14]$

10. final Result will be 14.

Output

Postfix Expression: $512 + 4 * + 3 -$

Result :14